Evaluación Parcial 2 — ISY0101 Ingeniería de Soluciones con IA

Agente de ahorro de TodoBarato: consulta, escritura y razonamiento con memoria y replanificación

Integrantes: Nicolas Estefania · Ariel Espinoza | Docente: [NOMBRE DEL DOCENTE] | Repositorio: github.com/niKo-ing/PROYECTOENGINEER-AI (rama master)

1. Caso y objetivo

TodoBarato es un comparador de precios de tecnología en Chile (catálogo, ofertas por tienda e historial de precios). El flujo automatizado es el de un asesor de compras: el cliente dice qué necesita y cuánto puede gastar, y el agente busca, compara tiendas, juzga con el historial si el precio está inflado y deja registro en la base de datos (una alerta de precio o una lista de compra). Además recuerda las preferencias del cliente entre conversaciones, y se usa desde la pantalla /agente del frontend (chat con los paneles «Cómo pensó» y «Guardado»). Se construyó sobre el proyecto de la evaluación anterior (catálogo, ofertas, perfil de usuario y capa de embeddings), que no se modificó.

2. Diseño e implementación del agente

El agente es un grafo de LangGraph (langgraph 1.2) con Gemini como modelo, conectado mediante langchain-google-genai. Se ejecuta detrás de un endpoint FastAPI (POST /api/v1/agent/run) y devuelve la respuesta, el plan, las herramientas usadas y la traza completa del turno.

Orquestación de componentes del agente de ahorro

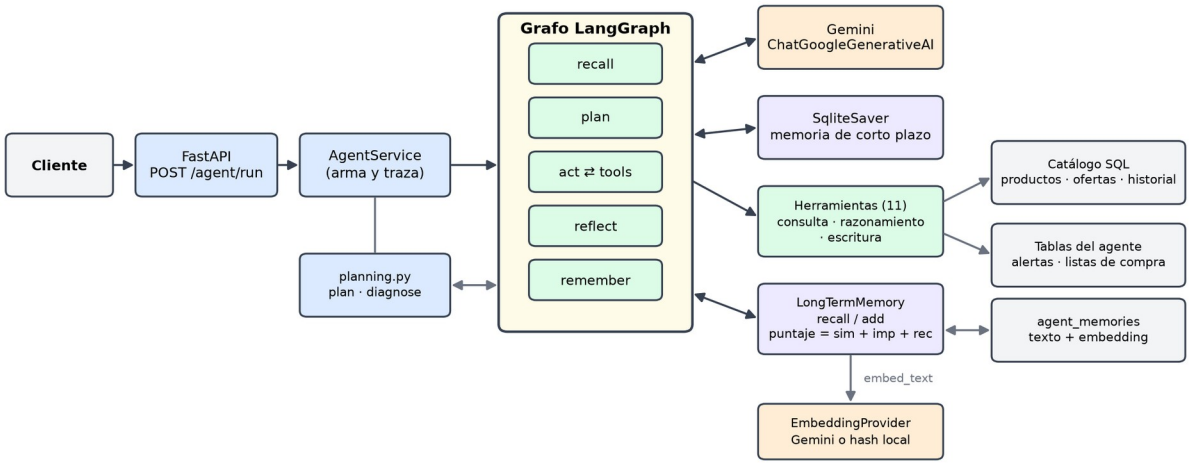


Figura 1. Orquestación de componentes.

Tipo	Herramienta	Función
Consulta	search_products, get_product_offers, get_price_history, compare_products, get_user_profile	Reutilizan los servicios del catálogo existentes
Consulta	recall_memory, check_price_alerts	Búsqueda semántica en memoria; revisa alertas contra los precios de hoy
Razonamiento	analyze_price_trend	Veredicto numérico: buen_momento / precio_normal / esperar, según mínimo y promedio históricos
Escritura	create_price_alert, save_shopping_list, remember_preference	Escriben en la base: alerta, lista con total vs. presupuesto, preferencia del usuario

Las herramientas validan sus argumentos con Pydantic y, ante un error de negocio, devuelven el error como observación (ToolException) en vez de abortar, de modo que el agente pueda reaccionar. Cada hilo de conversación se ata al usuario (thread_id = usuario:hilo) y los fallos del proveedor del modelo se traducen a 429/503 en lugar de un error de servidor. Las herramientas se ejecutan de a una porque comparten una sola sesión de base de datos, y guardar una lista de compra es una regla en código: solo ocurre si el pedido la menciona. Las tablas del agente y las de su historial se crean con RLS activado, porque Supabase expone el esquema público por su API.

3. Memoria y recuperación de contexto

- **Corto plazo:** el historial de cada hilo vive en un checkpointer de LangGraph (SqliteSaver en local; PostgresSaver cuando la app usa Postgres, probado con dos conexiones sobre un Postgres 16 real): sobrevive a reinicios del proceso y permite retomar una tarea. Se guarda una sola vez al terminar cada tarea: con ~150 ms por consulta a Supabase, guardar en cada paso escribía 188 filas por respuesta y superaba los 30 s del proxy; ahora son 7. Al modelo solo se le envía una ventana de los últimos 24 mensajes (sin dejar un ToolMessage huérfano) más un resumen determinista de lo recortado.
- **Largo plazo:** tabla agent_memories (preferencias, hechos y episodios de cada tarea) con embedding. La recuperación combina memoria núcleo (las preferencias de importancia ≥ 4 se cargan siempre) y búsqueda semántica con puntaje 0,75·similitud + 0,15·importancia + 0,10·recencia (vida media 45 días). No guarda duplicados (similitud ≥ 0,93; una tarea repetida actualiza su episodio en vez de agregar otro), está aislada por usuario y, si falla el embedding, busca por palabras.

Verificación: una preferencia declarada en un hilo (“Prefiero Samsung y no quiero Huawei”) aparece en la memoria usada por un hilo nuevo (“Recomiéndame un celular”) aunque no compartan palabras. La prueba opt-in contra Gemini real confirma que el ranking por embeddings acierta sin coincidencia de palabras (“laptop económico para estudiar” → “Prefiero notebooks baratos para la universidad”).

4. Planificación y toma de decisiones adaptativas

El nodo plan pide al LLM un plan JSON con prioridades 1–3; si la respuesta es inválida se usa un plan por reglas, de modo que el agente nunca queda sin plan. Tras cada ronda de herramientas, el nodo reflect aplica reglas deterministas (diagnose) y decide si replanificar:

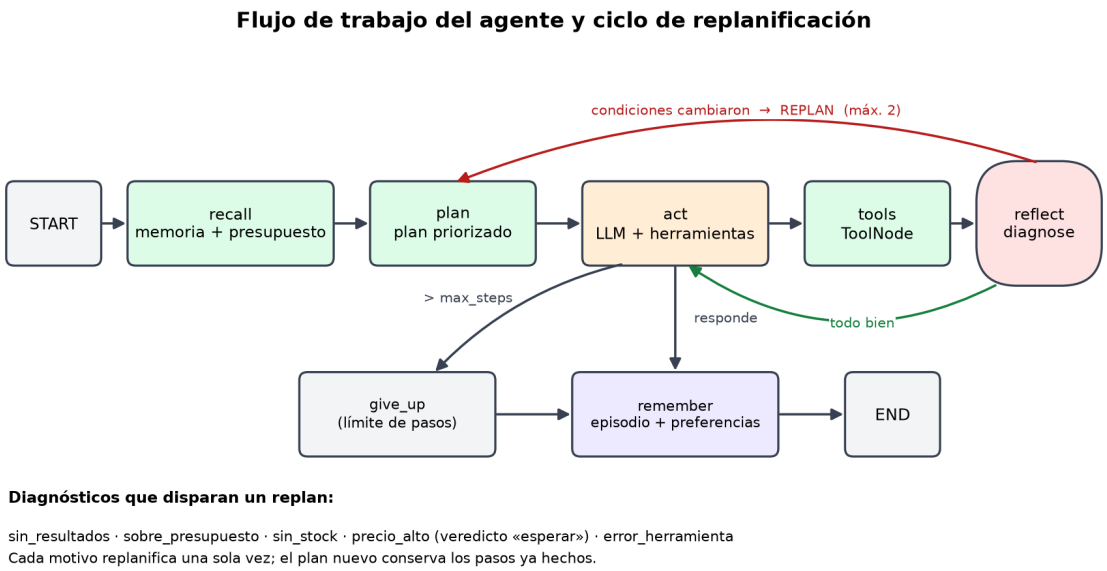


Figura 2. Flujo del grafo y ciclo de replanificación.

Observación	Diagnóstico	Adaptación
Búsqueda con 0 resultados	sin_resultados	Repetir con términos más generales y sin filtro de precio
Todo cuesta más que el presupuesto	sobre_presupuesto	Gama inferior; si no hay, alerta de precio e informar cuánto falta
Todas las ofertas sin stock	sin_stock	Descartar y elegir otra tienda o candidato
Veredicto de tendencia «esperar»	precio_alto	No comprar hoy; alerta cerca del promedio histórico
Herramienta devolvió error	error_herramienta	Corregir parámetros y reintentar una vez

Dos frenos evitan bucles: máximo 2 replanificaciones por pedido y cada motivo replanifica una sola vez. Un replan conserva los pasos ya hechos y no los duplica. Un tope de 10 pasos cierra las llamadas pendientes y responde con lo logrado.

5. Pruebas y evidencia

Evidencia	Resultado
Suite de pruebas del agente (tests/agent)	129 pruebas (124 pasan; 5 son opt-in contra Gemini y Postgres reales): planificación, memoria, herramientas, grafo, API, RLS y limpieza de respuestas. Suite completa del repositorio: 909 pasan
Escenarios de extremo a extremo	Compra con lista guardada; replan sin_resultados → sobre_presupuesto con alerta; precio_alto → alerta en vez de lista; sin_stock; error de herramienta; tope de pasos; memoria entre hilos y usuarios; reinicio del proceso; alerta que se cumple en un turno posterior
Demo guionada (scripts/demo_agente.py)	5 historias con traza completa, determinista y sin red: docs/agente/evidencia/demo_guionada.txt
Demo con Gemini real (--live)	Historias 1 a 3 completas con el mismo comportamiento adaptativo (replan por sin_resultados, sobre_presupuesto y precio_alto; alertas y lista creadas); la cuota gratuita se agotó en la historia 4 y el agente lo informó sin caerse: docs/agente/evidencia/demo_gemini.txt
Pruebas en vivo (tests/agent/test_live_gemini.py)	2 pasan: ranking semántico con embeddings reales y tarea

	de compra completa dentro del presupuesto
Pantalla /agente con Gemini y el catálogo real de Supabase	Validadas con capturas (docs/agente/evidencia/real_*.png): alerta con replan, revisión de alertas, memoria entre conversaciones y comparación de tiendas
Defectos hallados con datos reales y corregidos	Sesión de base compartida entre herramientas paralelas; historial lento por la latencia (188 → 7 filas); tablas del historial sin RLS; listas guardadas sin pedirlas (regla en código); episodios duplicados

Un hallazgo de la evidencia en vivo: Gemini escribe planes más escuetos que el plan por reglas (a veces solo nombres de herramientas) y, aun así, el agente completó las tareas. En las pruebas con el modelo guionado, este es un doble determinista que reacciona a las observaciones: sirve para probar el grafo, no la calidad de Gemini; esa se contrastó con la ejecución en vivo.

6. Justificación de la elección de componentes

- **LangGraph:** el flujo no es lineal: después de cada ronda de herramientas el agente puede volver a planificar (reflect → plan), necesita un estado compartido y un checkpointer persistente. El orquestador que ya existía en el proyecto era sin estado entre llamadas (el cliente reenviaba el historial), lo que no sirve para tareas largas. LangGraph entrega ciclo, estado tipado, ToolNode y checkpointers listos, y se integra con Gemini mediante langchain-google-genai, el mismo proveedor y las mismas variables GEMINI_* del resto de la app (IE2: compatibilidad).
- **Reglas deterministas para decidir:** las decisiones que no deben depender de que el modelo acierte (diagnóstico de la observación, veredicto comprar/esperar con mínimo y promedio históricos, extracción del presupuesto) son código. Así se prueban sin red y sin gastar cuota (más de 120 pruebas), y el LLM queda para lo que sí le corresponde: proponer el plan, elegir herramientas y redactar.
- **Checkpointer:** SQLite en desarrollo y Postgres cuando la app usa Postgres (AGENT_CHECKPOINT=auto), probado con dos conexiones sobre un Postgres 16 real; es lo que permite escalar a varias réplicas sin perder los hilos (IE2: escalabilidad).
- **Memoria de largo plazo:** se reutilizó el EmbeddingProvider del RAG del proyecto. El puntaje combina similitud, importancia y recencia (idea de Park et al., 2023). La memoria núcleo (preferencias importantes siempre cargadas, idea de MemGPT) se agregó por una falla medida en nuestra propia demo: con embeddings locales, «Recomiéndame un celular» no recuperaba «Prefiero Samsung» porque no comparten palabras.
- **Herramientas de escritura:** el flujo organizacional exige dejar registro, no solo responder: alertas y listas persisten en la base, con argumentos validados por Pydantic y errores devueltos como observación para poder replanificar.
- **Límites conocidos:** el progreso del plan es aproximado (un paso cuenta como hecho si nombra una herramienta ejecutada), las alertas se evalúan cuando el usuario las revisa y no en segundo plano, la cuota gratuita de Gemini limitó la demo en vivo, y sin llave de Gemini los embeddings locales solo coinciden por palabras (la memoria núcleo lo compensa).

7. Declaración de uso de inteligencia artificial

Se usó Claude Code (Anthropic, modelo Claude Sonnet 5.5) como apoyo para: implementar el código del agente y sus pruebas, generar los diagramas con matplotlib, ejecutar las demos, redactar la documentación técnica (README, arquitectura y bocetos de diseño), armar la presentación, construir la

pantalla del agente en el frontend y redactar un borrador de las secciones 6 (justificación) y 8 (conclusiones y reflexiones) de este informe. El equipo revisó y ejecutó el código, las pruebas y las demos, y validó que el contenido coincida con lo implementado. Las reflexiones individuales son borradores que cada integrante debe ajustar a su aporte real. Cita (APA, según bibliotecas.duoc.cl/ia): Anthropic. (2026). Claude Sonnet 5.5 [Modelo de lenguaje de gran tamaño]. https://claude.ai

8. Conclusiones y reflexión individual

Conclusiones del equipo. Se cumplió el objetivo: el agente integra consulta, razonamiento y escritura, y resuelve el flujo de asesoría de compras de punta a punta con registro en la base. Cuatro hallazgos medidos: (1) Gemini escribe planes más escuetos que el plan por reglas y aun así completó las tareas, por lo que el diagnóstico determinista fue lo que garantizó la adaptación; (2) los embeddings locales no capturan significado, lo que obligó a agregar la memoria núcleo; (3) separar el modelo guionado de las pruebas en vivo permitió verificar el grafo sin depender de la cuota; (4) probar con el proveedor y la base reales destapó defectos que el modelo guionado no mostraba (sesión compartida entre herramientas paralelas, latencia del historial, tablas sin RLS, escrituras no pedidas), por lo que las pruebas deterministas se complementan con validación real. Como siguiente paso queda evaluar las alertas en segundo plano tras cada ingesta.

Reflexión de Nicolas Estefania (borrador; ajustar a su aporte real). Aprendí a diseñar un agente como un grafo con estado y a decidir qué va en código y qué en el modelo. Lo más valioso fue ver que una falla (la memoria que no recuperaba la preferencia) se resolvía con una decisión de diseño y no con un prompt. Mi contribución fue [COMPLETAR: p. ej. el grafo, las herramientas, las pruebas].

Reflexión de Ariel Espinoza (borrador; ajustar a su aporte real). Aprendí a probar un sistema con LLM: separar un doble determinista para el grafo de las pruebas en vivo y medir en lugar de suponer. Entendí por qué la memoria de largo plazo necesita importancia y recencia además de similitud. Mi contribución fue [COMPLETAR: p. ej. la memoria, la documentación, la demo].

9. Referencias

Google. (s. f.). Gemini API: Embeddings. https://ai.google.dev/gemini-api/docs/embeddings

LangChain. (s. f.). LangGraph overview. https://langchain-ai.github.io/langgraph/

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. arXiv. https://arxiv.org/abs/2310.08560

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. En Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23). ACM. https://doi.org/10.1145/3586183.3606763

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR 2023. https://arxiv.org/abs/2210.03629